



GUÍA PARA LA DEFENSA

PULSO

Código, arquitectura, ejemplos y preguntas para estudiar

De la vista general al detalle técnico

Pensado para leer, explicar y cargar en
NotebookLM

Proyecto académico · PHP · MySQL · HTML · CSS · JavaScript

Cómo usar esta guía

Este documento está construido sobre los archivos reales de la entrega. Combina una lectura superficial para ubicarse, una lectura funcional para seguir el viaje de los datos y una lectura técnica para responder preguntas específicas de la defensa.

Nivel	Qué conviene dominar	Resultado esperado
1. Panorama	Propósito, módulos y tecnologías.	Poder explicar PULSO en 30 segundos.
2. Flujo	Navegador -> PHP -> MySQL -> HTML.	Seguir un alta, una consulta y una edición.
3. Detalle	GET/POST, JOIN, claves, validación y escape.	Responder el porqué de cada bloque importante.
4. Revisión crítica	Riesgos, módulos incompletos y mejoras.	Defender lo hecho sin ocultar las limitaciones.

Verificación realizada

Los 20 archivos PHP pasaron el chequeo de sintaxis con el intérprete local. Esto confirma que PHP puede analizarlos, pero no demuestra por sí solo que todos los enlaces, consultas o flujos funcionen correctamente.

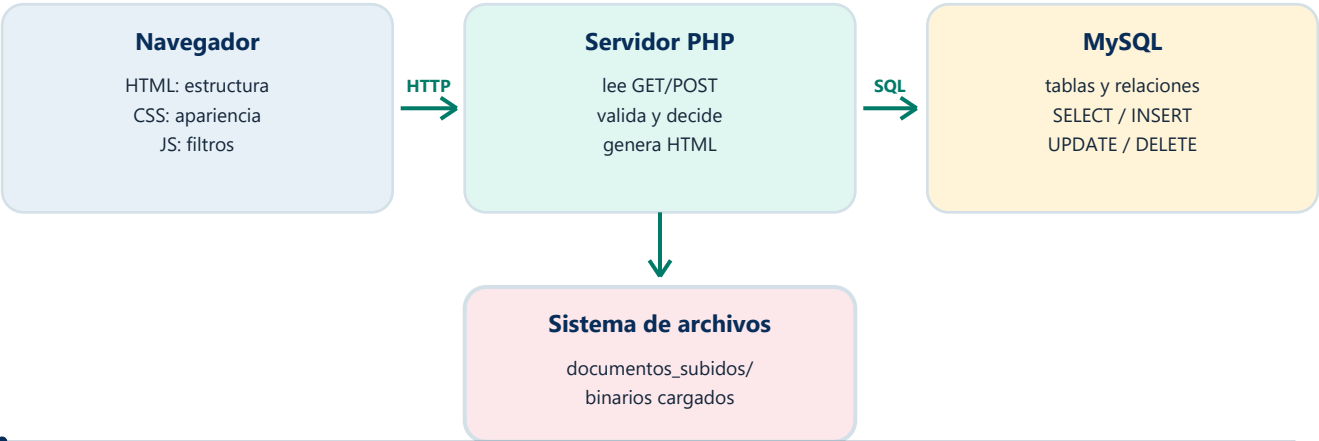
Índice

Cómo usar esta guía	2
Índice	3
1. PULSO en una mirada	5
Tecnologías y responsabilidad	5
Estado real de los módulos	6
Mapa de archivos	6
2. Capa visible: HTML y CSS	7
HTML: estructura semántica	7
CSS: una identidad visual reutilizable	7
3. PHP y conexión a MySQL	8
La puerta de entrada a la base	8
GET y POST	8
4. Cómo viajan los datos	9
Por qué se redirige después de guardar	9
Tres validaciones diferentes	9
5. Caso principal: pacientes	10
5.1 Listar: SELECT + JOIN	10
5.2 Mostrar sin ejecutar código guardado	10
5.3 Alta: dos INSERT relacionados	11
5.4 Ficha y edición	12
5.5 Baja: el problema más importante	13
5.6 Filtro con JavaScript	14
6. Modelo de datos	15
Claves y restricciones	15
Persona y paciente: normalización	16
Relación muchos a muchos	16
7. Ambulancias	17
Listado y catálogo de estados	17
Alta	17
Por qué la matrícula no se edita	18
8. Documentos: base de datos + archivos	18

Actualizar o eliminar un documento	19
Qué significa bind_param('ss', ...)	19
9. Equipamientos, funcionarios y traslados	20
Equipamientos: un buen uso de consultas preparadas	20
Funcionarios: inconsistencias concretas	21
Traslados: modelo ambicioso, interfaz pendiente	22
Otras inconsistencias de navegación	22
10. Seguridad y calidad técnica	23
Consulta directa vs. consulta preparada	23
Plan de mejora priorizado	24
Qué significa que pase el chequeo de sintaxis	24
11. Ejemplos para explicar en voz alta	25
Ejemplo A: registrar un paciente	25
Ejemplo B: buscar 'critico'	25
Ejemplo C: cargar protocolo.pdf	25
Ejemplo D: abrir una ambulancia	25
12. Guion breve para la defensa	26
Demostración sugerida	26
Si algo falla durante la demo	26
13. Preguntas probables y respuestas	27
14. Banco de autoevaluación	28
Clave breve	28
15. Prompts sugeridos para NotebookLM	29
16. Glosario y machete final	30
Machete de una página	31
Archivos fuente revisados	31

1. PULSO en una mirada

PULSO es un sistema web académico para registrar y consultar información operativa vinculada con pacientes, ambulancias, documentos, funcionarios, equipamientos y traslados. No usa un framework: cada página PHP combina lógica del servidor, consultas SQL y la vista HTML.



Explicación de 30 segundos

El navegador muestra formularios y listados. Cuando una acción necesita datos, PHP recibe la solicitud, valida lo enviado, consulta o modifica MySQL y devuelve HTML. JavaScript mejora algunos filtros sin recargar. Los archivos de documentos se guardan en una carpeta y su ruta queda registrada en la base.

Tecnologías y responsabilidad

Tecnología	Responsabilidad en PULSO	Ejemplo real
HTML	Estructura de páginas, formularios, tablas y navegación.	panel.html y la parte visual de los .php.
CSS	Colores, tamaños, grillas, botones, tarjetas y estados.	estilos.css.
JavaScript	Filtrado instantáneo en listados.	Funciones normalizar() y actualizarListado().
PHP	Validación, reglas, consultas y generación de HTML.	Altas, fichas, edición y eliminación.
MySQL	Persistencia, claves, relaciones y valores predeterminados.	pulso_base_de_datos.sql.

Estado real de los módulos

Para defender el proyecto conviene distinguir entre una pantalla existente y un ABM completo. ABM significa alta, baja y modificación; habitualmente también se incluye consulta o listado.

Módulo	Qué existe	Estado para la defensa
Pacientes	Listado, filtro, alta, ficha, edición y eliminación.	Es el flujo más didáctico. La baja tiene un supuesto incorrecto entre identificadores.
Ambulancias	Listado, alta, ficha, edición y eliminación.	Bastante completo. Usa matrícula como clave.
Documentos	Listado, carga física, edición/reemplazo y eliminación.	Completo y más seguro en SQL porque usa consultas preparadas.
Equipamientos	Listado y alta.	Parcial: faltan ficha, edición, eliminación y asignación visual a ambulancias.
Funcionarios	Listado, filtro y formulario de alta.	Incompleto: el formulario y los enlaces tienen inconsistencias que impiden cerrar el flujo.
Traslados	Esquema SQL y una vista de módulo pendiente.	No hay ABM conectado a la base.
Acceso	Pantalla de entrada con enlace al panel.	No implementa autenticación ni sesiones.

Frase segura para la defensa

No todos los módulos tienen el mismo grado de avance. El núcleo demostrable está en pacientes, ambulancias y documentos; equipamientos y funcionarios están parciales, y traslados quedó modelado en la base pero pendiente en la interfaz.

Mapa de archivos

Familia	Archivos principales	Patrón
Comunes	conexion.php, estilos.css, panel.html	Conexión, diseño y navegación.
Pacientes	pacientes.php, nuevo-*, ficha-*, editar-*, eliminar_*	Un archivo por caso de uso.
Ambulancias	Listado, alta, ficha, edición y baja.	Repite el patrón de pacientes.
Documentos	Listado, cargar, editar y eliminar.	Combina base de datos y archivos físicos.
Base	pulso_base_de_datos.sql	Crea tablas, relaciones, catálogos y datos de prueba.

2. Capa visible: HTML y CSS

HTML: estructura semántica

Las páginas usan etiquetas como `<header>`, `<nav>`, `<main>`, `<section>`, `<form>` y `<table>`. Estas etiquetas describen la función de cada zona y ayudan a la accesibilidad.

```

panel.html - cabecera y navegación | líneas 13-28

13     
14     <span>PULSO</span>
15     </a>
16     <nav class="navegacion" aria-label="Páginas principales">
17         <a class="enlace-activo" href="panel.html">Panel</a>
18         <a href="pacientes.php">Pacientes</a>
19         <a href="funcionario.php">Funcionarios</a>
20         <a href="documentos.php">Documentos</a>
21         <a href="ambulancia.php">Ambulancias</a>
22         <a href="equipamientos.php">Equipamientos</a>
23         <a href="index.html">Inicio</a>
24     </nav>
25 </header>
26
27 <main class="pagina">
28     <section class="encabezado-pagina">

```

Elemento	Qué hace	Qué decir
<code></code>	Navega a otro archivo o caso de uso.	El proyecto se organiza como páginas enlazadas.
<code><form method='post'></code>	Envía datos al servidor.	POST se usa cuando la acción modifica datos.
<code><input name=...></code>	El atributo name se convierte en una clave de <code>\$_POST</code> .	Sin name, el campo no llega a PHP.
required	Valida en el navegador.	Ayuda al usuario, pero PHP debe volver a validar.

CSS: una identidad visual reutilizable

```

estilos.css - variables globales | líneas 1-17

1  :root {
2      --azul: #082d58;
3      --azul-claro: #e7f0f7;
4      --verde: #00a58a;
5      --verde-oscuro: #007b68;
6      --verde-claro: #dfff6f1;
7      --texto: #17324d;
8      --texto-suave: #5c7184;
9      --borde: #d3e0e7;
10     --fondo: #f1f7f8;
11     --blanco: #ffffff;
12     --amarillo-claro: #fff0d2;
13     --amarillo-texto: #80530d;
14     --rojo-claro: #fce8eb;
15     --rojo-texto: #8b2430;
16     --gris-claro: #e9eef1;
17     --sombra: 0 12px 28px rgba(8, 45, 88, 0.08);

```

Las variables de `:root` centralizan la paleta. Por ejemplo, si cambia `--verde`, cambian botones y acentos que reutilizan esa variable. Clases como `.tarjeta`, `.boton`, `.tabla` y `.estado` evitan repetir estilos.

Detalle útil

La tabla tiene un ancho mínimo y vive dentro de un contenedor con desplazamiento horizontal. Eso evita que se aplaste en pantallas pequeñas, aunque el CSS no contiene media queries y la adaptación móvil es parcial.

3. PHP y conexión a MySQL

La puerta de entrada a la base

conexion.php | líneas 3-19

```

3 // Datos que PHP necesita para encontrar la base de datos local de XAMPP.
4 $servidor = "localhost";
5 $usuario = "root";
6 $contrasena = "";
7 $base_datos = "pulso_entrega";
8
9 // Hace que MySQL lance excepciones; permite controlar los errores con try/catch.
10 mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
11
12 try {
13     // Abre la conexión y configura UTF-8 para guardar correctamente tildes y eñes.
14     $conexion = new mysqli($servidor, $usuario, $contrasena, $base_datos);
15     $conexion->set_charset("utf8mb4");
16 } catch (mysqli_sql_exception $error) {
17     // Evita mostrar datos técnicos de la base si la conexión falla.
18     http_response_code(500);
19     die("No se pudo conectar con la base de datos.");

```

Código	Significado técnico	En palabras simples
<code>mysqli_report(...)</code>	Hace que errores MySQLi se conviertan en excepciones.	Permite atraparlos con try/catch.
<code>new mysqli(...)</code>	Abre una conexión al servidor y base seleccionados.	PHP se comunica con MySQL.
<code>set_charset('utf8mb4')</code>	Define la codificación de la conexión.	Conserva tildes, eñes y otros caracteres.
<code>require_once</code>	Incluye el archivo una sola vez.	Cada página reutiliza la misma conexión.
<code>http_response_code(500)</code>	Devuelve un estado HTTP de error interno.	El navegador recibe una señal correcta de falla.

Alcance académico

Las credenciales están escritas en el archivo y corresponden a una instalación local de XAMPP: usuario root, contraseña vacía y base pulso_entrega. En producción se guardarían fuera del código, con un usuario de permisos limitados.

GET y POST

Método	Uso observado	Ejemplo
GET	Identifica qué ficha o edición abrir. El dato viaja en la URL.	<code>ficha-paciente.php?id=3</code>
POST	Envía formularios que crean, actualizan o eliminan.	El alta lee <code>\$_POST['nombres']</code> .

ficha-paciente.php - validar un id GET | líneas 5-12

```

5 // El id llega en la URL, por ejemplo: ficha-paciente.php?id=3.
6 $idPaciente = filter_input(INPUT_GET, "id", FILTER_VALIDATE_INT);
7
8 if ($idPaciente === false || $idPaciente === null || $idPaciente < 1) {
9     http_response_code(400);
10    die("El identificador del paciente no es válido.");
11 }
12

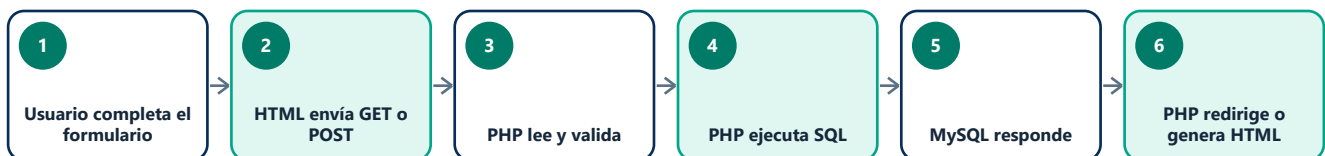
```

Regla de defensa

GET no significa inseguro y POST no significa seguro. La seguridad depende de validar, autorizar, proteger contra CSRF y parametrizar SQL. Aquí POST se usa correctamente para las bajas, pero faltan varias de esas capas.

4. Cómo viajan los datos

El patrón central se repite en casi todos los módulos. Entender esta secuencia vale más que memorizar cada línea.



Por qué se redirige después de guardar

Después de un INSERT o UPDATE aparece `header('Location: ...')` seguido de `exit`. Este patrón se conoce como POST/Redirect/GET: evita que actualizar el navegador repita el envío y, por ejemplo, cree el mismo registro otra vez.

```

nuevo-paciente.php - final del alta | líneas 55-66

55      // MySQL completa fecha_registro y activo con sus valores predeterminados.
56      $conexion->query(
57          "INSERT INTO paciente
58              (id_persona, estado, patologia)
59              VALUES
60              ($idPersona, '$estado', '$patologia')"
61      );
62
63      // Vuelve al listado para mostrar el nuevo paciente.
64      header("Location: pacientes.php");
65      exit;
66  } catch (mysqli_sql_exception $error) {
  
```

Tres validaciones diferentes

Capa	Ejemplos	Qué evita
Navegador	<code>required, type='email', min='1'</code> .	Errores simples y mejora la experiencia.
PHP	Campos no vacíos, enteros válidos, listas permitidas.	Datos manipulados o incompletos que saltan HTML.
MySQL	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL.	Inconsistencias que lleguen hasta la base.

Idea clave

Las tres capas se complementan. El atributo `required` no reemplaza la validación de PHP, y la validación de PHP no reemplaza las restricciones de la base.

5. Caso principal: pacientes

5.1 Listar: SELECT + JOIN

```

pacientes.php - consulta del listado | líneas 3-21

3  require_once "conexion.php";
4
5
6  $resultado = $conexion->query(
7      "SELECT
8          paciente.id_paciente,
9          DATE_FORMAT(paciente.fecha_registro, '%d/%m/%Y %H:%i') AS fecha_registro,
10         paciente.estado,
11         paciente.patologia,
12         paciente.activo,
13         persona.ci,
14         persona.nombres,
15         persona.apellidos,
16         persona.telefono
17     FROM paciente
18     INNER JOIN persona ON persona.id_persona = paciente.id_persona
19     ORDER BY paciente.id_paciente DESC"
20 );
21

```

Los datos de una persona están separados de los datos clínicos. Por eso la consulta necesita un `INNER JOIN`: combina cada fila de paciente con la fila de persona que comparte `id_persona`.

Parte SQL	Interpretación
<code>SELECT</code>	Elige las columnas que regresarán.
<code>FROM paciente</code>	Define la tabla inicial.
<code>INNER JOIN persona ON ...</code>	Solo devuelve relaciones coincidentes.
<code>DATE_FORMAT</code>	Convierte la fecha a un formato legible para la interfaz.
<code>ORDER BY ... DESC</code>	Muestra primero los pacientes con id más alto, normalmente los últimos.
<code>fetch_all(MYSQLI_ASSOC)</code>	Convierte el resultado en un arreglo con nombres de columnas.

5.2 Mostrar sin ejecutar código guardado

```

pacientes.php - función escapar | líneas 24-29

24
25 function escapar($valor)
26 {
27     return htmlspecialchars($valor ?? "", ENT_QUOTES, "UTF-8");
28 }
29

```

`htmlspecialchars` transforma caracteres especiales antes de insertarlos en HTML. Si un nombre contuviera `<script>`, el navegador lo mostraría como texto en vez de interpretarlo. Esta defensa actúa al mostrar; no protege la consulta SQL.

5.3 Alta: dos INSERT relacionados

```
nuevo-paciente.php - lectura y validación | líneas 6-34

6  if ($_SERVER["REQUEST_METHOD"] === "POST") {
7      require_once "conexion.php";
8
9      // Lee los campos enviados; trim quita espacios al principio y al final.
10     $nombres = trim($_POST["nombres"] ?? "");
11     $apellidos = trim($_POST["apellidos"] ?? "");
12     $ci = trim($_POST["ci"] ?? "");
13     $telefono = trim($_POST["telefono"] ?? "");
14     $fechaNacimiento = trim($_POST["fecha_nacimiento"] ?? "");
15     $sexo = trim($_POST["sexo"] ?? "");
16     $estado = trim($_POST["estado"] ?? "");
17     $direccion = trim($_POST["direccion"] ?? "");
18     $patologia = trim($_POST["patologia"] ?? "");
19     $email = trim($_POST["email"] ?? "");
20
21     // Listas cerradas para impedir valores que no existen en los select del formulario.
22     $sexosValidos = ["masculino", "femenino", "otro"];
23     $estadosValidos = ["estable", "en_revision", "critico"];
24
25     // PHP vuelve a validar lo obligatorio aunque el navegador ya use required.
26     if (
27         $nombres === "" || $apellidos === "" || $ci === "" ||
28         !in_array($estado, $estadosValidos, true) ||
29         ($sexo !== "" && !in_array($sexo, $sexosValidos, true))
30     ) {
31         $mensajeError = "Complete correctamente todos los campos obligatorios.";
32     } else {
33         try {
34             // MySQL espera NULL, no una cadena vacía, cuando no se informa una fecha.
```

El alta lee los campos con `$_POST`, elimina espacios con `trim` y limita sexo y estado a listas conocidas. La fecha vacía se convierte en `NULL` porque una fecha desconocida no es lo mismo que una cadena vacía.

```
nuevo-paciente.php - persona, insert_id y paciente | líneas 43-62

43     // Guarda primero los datos personales.
44     $conexion->query(
45         "INSERT INTO persona
46         (nombres, apellidos, ci, telefono, fecha_nacimiento, sexo, direccion, email)
47         VALUES
48         ('$nombres', '$apellidos', '$ci', '$telefono',
49         '$fechaNacimientoParaSQL', '$sexo', '$direccion', '$email')
50     );
51
52     // Recupera el id generado para relacionar ambas tablas.
53     $idPersona = $conexion->insert_id;
54
55     // MySQL completa fecha_registro y activo con sus valores predeterminados.
56     $conexion->query(
57         "INSERT INTO paciente
58         (id_persona, estado, patologia)
59         VALUES
60         ($idPersona, '$estado', '$patologia')
61     );
62
```



Por qué insert_id es esencial

Persona y paciente tienen contadores independientes. insert_id devuelve la clave creada por el primer INSERT para que paciente apunte a la persona correcta. No se debe adivinar ni asumir que los identificadores coinciden.

Ejemplo trabajado

Entrada	Valor de ejemplo
Nombre	Ana Silva
CI	4.567.890-1
Estado	critico
Patología	Dolor torácico

Resultado conceptual: MySQL crea una fila en persona, por ejemplo con id_persona = 12. Luego crea una fila en paciente con id_persona = 12. La fecha de registro y activo = TRUE aparecen por valores DEFAULT del esquema.

5.4 Ficha y edición

La ficha valida el id recibido, consulta un único paciente y responde con 400 si el identificador no es válido o 404 si no existe. La edición primero recupera la relación con persona y luego actualiza las dos tablas.

```

editar-pacientes.php - dos UPDATE | líneas 70-90

70         ? "NULL"
71         : "'$fechaNacimiento'";
72     $emailParaSQL = $email === "" ? "NULL" : "'$email'";
73     $idPersona = $paciente["id_persona"];
74
75     $conexion->query(
76         "UPDATE persona
77         SET nombres = '$nombres', apellidos = '$apellidos', ci = '$ci',
78         fecha_nacimiento = '$fechaNacimientoParaSQL', sexo = '$sexo',
79         telefono = '$telefono', direccion = '$direccion', email = $emailParaSQL
80         WHERE id_persona = $idPersona"
81     );
82
83     $conexion->query(
84         "UPDATE paciente
85         SET estado = '$estado', patologia = '$patologia'
86         WHERE id_paciente = $idPaciente"
87     );
88
89     header("Location: ficha-paciente.php?id=" . $idPaciente . "&actualizado=ok");
90     exit;

```

Decisión	Motivo
Actualizar persona y paciente	Los datos personales y clínicos viven en tablas diferentes.
No cambiar id_persona	Es la identidad de la relación, no un dato editable.
Redirigir a la ficha	Permite ver inmediatamente el estado persistido.
Agregar estado alta	La edición admite cerrar el episodio aunque el alta inicial no lo ofrezca.

5.5 Baja: el problema más importante

eliminar_paciente.php - supuesto incorrecto | líneas 17-26

```
17
18 $conexion->query(
19     "DELETE FROM paciente WHERE id_paciente = $idPaciente"
20 );
21
22 // Por ahora el proyecto supone que id_paciente e id_persona tienen el mismo valor.
23 $conexion->query(
24     "DELETE FROM persona WHERE id_persona = $idPaciente"
25 );
26
```

Qué está mal

El código borra paciente por id_paciente y luego intenta borrar persona usando ese mismo número. El esquema solo garantiza la relación mediante paciente.id_persona; no garantiza que id_paciente sea igual a id_persona. Con el tiempo ambos contadores pueden divergir y se podría dejar una persona huérfana o intentar borrar otra.

La solución conceptual es consultar primero id_persona, ejecutar ambas eliminaciones dentro de una transacción y confirmar solo si las dos terminan bien. Otra alternativa es una baja lógica con activo = FALSE, que ya está modelada pero no se usa al eliminar.

Estrategia	Ventaja	Costo o condición
Baja física	Elimina definitivamente las filas.	Debe respetar FK, recuperar id_persona real y usar transacción.
Baja lógica	Conserva historial y permite reactivar.	Los listados deben filtrar o marcar activo = FALSE.

5.6 Filtro con JavaScript

pacientes.php - filtro en el navegador | líneas 145-177

```

123 const busquedaPaciente = document.getElementById("busqueda-paciente");
124 const filtroEstado = document.getElementById("filtro-estado");
125 const filas = document.querySelectorAll("#tabla-pacientes tbody tr[data-estado]");
126 const contador = document.getElementById("contador");
127 const etiquetaResultados = document.getElementById("etiqueta-resultados");
128
129 // Ignora mayúsculas y tildes para que la búsqueda sea más tolerante.
130 function normalizar(texto) {
131     return texto.normalize("NFD").replace(/[\u0300-\u036f]/g, "").toLowerCase();
132 }
133
134 // Oculta las filas que no coinciden y actualiza el contador visible.
135 function actualizarListado() {
136     const texto = normalizar(busquedaPaciente.value.trim());
137     const estado = filtroEstado.value;
138     let visibles = 0;
139
140     filas.forEach((fila) => {
141         const coincideTexto = texto === "" || normalizar(fila.textContent).includes(texto);
142         const coincideEstado = estado === "" || fila.dataset.estado === estado;
143         fila.hidden = !coincideTexto || !coincideEstado;
144         if (!fila.hidden) visibles += 1;
145     });
146
147     contador.textContent = visibles;
148     etiquetaResultados.textContent = visibles === 1 ? "resultado" : "resultados";
149 }
150
151 // Filtra en esta misma página; preventDefault evita una recarga innecesaria.
152 formularioFiltros.addEventListener("submit", (evento) => {
153     evento.preventDefault();
154     actualizarListado();
155 });

```

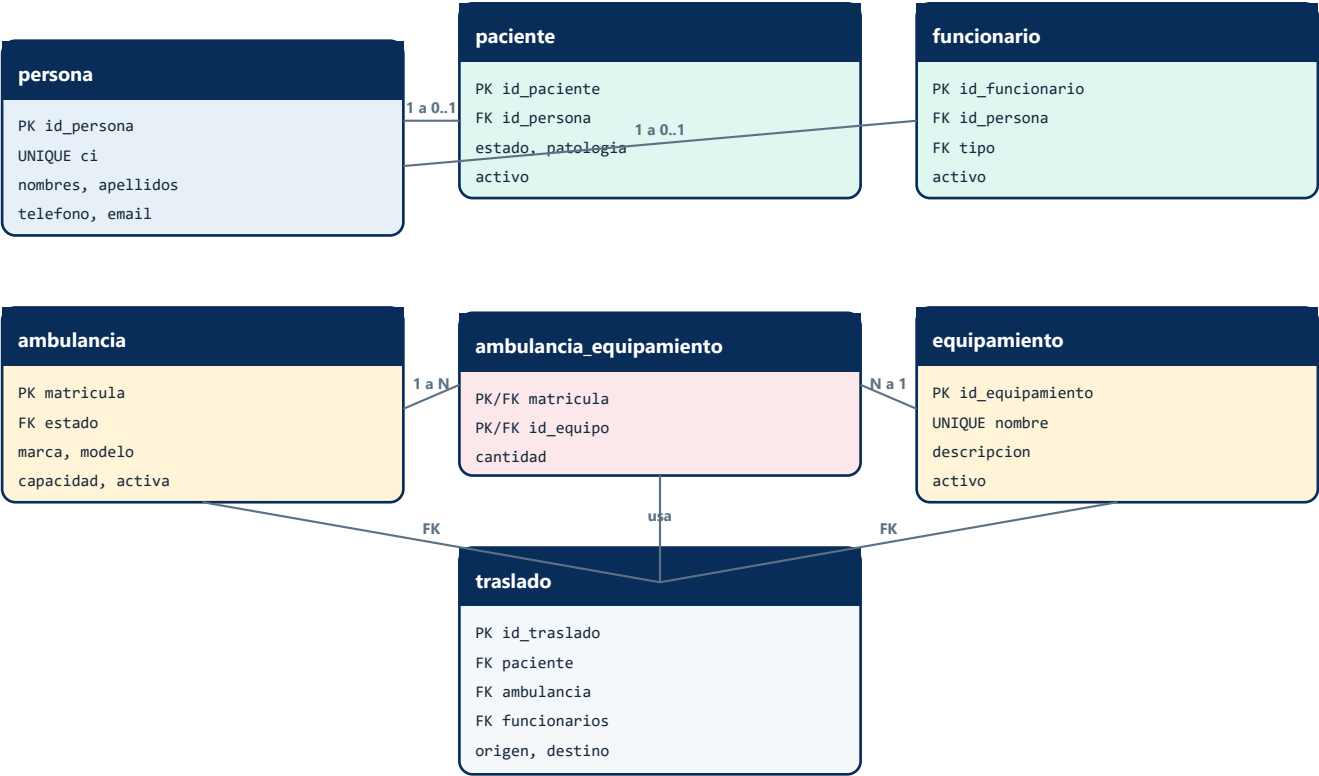
Paso	Qué ocurre
Tomar referencias	Se obtienen el formulario, los campos, las filas y el contador.
Normalizar	Se quitan tildes y se pasa a minúsculas.
Comparar	Se evalúa texto y estado para cada fila.
Ocultar	<code>fila.hidden</code> cambia la visibilidad sin modificar la base.
Evitar envío	<code>preventDefault()</code> impide recargar la página.

Límite del filtro

El filtro actúa solo sobre las filas que PHP ya cargó. Es ideal para pocos registros; con miles convendría enviar parámetros al servidor y paginar la consulta SQL.

6. Modelo de datos

El archivo principal crea la base pulso_entrega, tablas de negocio, catálogos y algunas relaciones. El siguiente diagrama resume las entidades más importantes; omite columnas secundarias para priorizar las relaciones.



Claves y restricciones

Concepto	Ejemplo PULSO	Función
PRIMARY KEY	paciente.id_paciente o ambulancia.matricula	Identifica una fila sin repetirse.
FOREIGN KEY	paciente.id_persona	Exige que la persona relacionada exista.
UNIQUE	persona.ci	Evita duplicar una cédula.
NOT NULL	ambulancia.marca	Obliga a guardar un valor.
DEFAULT	paciente.activo = TRUE	Completa automáticamente si PHP no envía el campo.
Clave compuesta	matrícula + equipamiento	Evita repetir el mismo equipo para una ambulancia.

Persona y paciente: normalización

pulso_base_de_datos.sql - relación persona/paciente | líneas 7-29

```

7 CREATE TABLE IF NOT EXISTS persona (
8     id_persona INT AUTO_INCREMENT,
9     ci VARCHAR(20) NOT NULL UNIQUE,
10    nombres VARCHAR(100) NOT NULL,
11    apellidos VARCHAR(100) NOT NULL,
12    fecha_nacimiento DATE,
13    sexo VARCHAR(20),
14    telefono VARCHAR(30),
15    direccion VARCHAR(255),
16    email VARCHAR(100),
17    PRIMARY KEY (id_persona)
18 ) ENGINE=InnoDB;
19
20 CREATE TABLE IF NOT EXISTS paciente (
21     id_paciente INT AUTO_INCREMENT,
22     id_persona INT NOT NULL UNIQUE,
23     fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
24     estado VARCHAR(30) NOT NULL DEFAULT 'estable',
25     patologia VARCHAR(255),
26     activo BOOLEAN NOT NULL DEFAULT TRUE,
27     PRIMARY KEY (id_paciente),
28     FOREIGN KEY (id_persona) REFERENCES persona(id_persona)
29 ) ENGINE=InnoDB;

```

Separar persona de roles como paciente y funcionario evita duplicar nombres, cédula, teléfono y domicilio. Técnicamente una persona podría tener un rol de cada tipo, porque ambos guardan un id_persona único.

Relación muchos a muchos

pulso_base_de_datos.sql - equipamientos por ambulancia | líneas 78-94

```

78 CREATE TABLE IF NOT EXISTS equipamiento (
79     id_equipamiento INT AUTO_INCREMENT,
80     nombre VARCHAR(100) NOT NULL UNIQUE,
81     descripcion VARCHAR(255),
82     activo BOOLEAN NOT NULL DEFAULT TRUE,
83     PRIMARY KEY (id_equipamiento)
84 ) ENGINE=InnoDB;
85
86 -- Resuelve la relación muchos a muchos entre ambulancias y equipos.
87 CREATE TABLE IF NOT EXISTS ambulancia_equipamiento (
88     matricula VARCHAR(20),
89     id_equipamiento INT,
90     cantidad INT NOT NULL DEFAULT 1,
91     PRIMARY KEY (matricula, id_equipamiento),
92     FOREIGN KEY (matricula) REFERENCES ambulancia(matricula),
93     FOREIGN KEY (id_equipamiento) REFERENCES equipamiento(id_equipamiento)
94 ) ENGINE=InnoDB;

```

Una ambulancia puede llevar varios equipos y un tipo de equipo puede estar en varias ambulancias. La tabla intermedia ambulancia_equipamiento resuelve esa relación N a N y agrega cantidad.

Versiones SQL

El proyecto contiene tres archivos SQL. El README identifica pulso_base_de_datos.sql como esquema principal. Para una entrega final conviene evitar que archivos numerados antiguos se importen por error.

7. Ambulancias

Listado y catálogo de estados

ambulance.php - estado por JOIN | lines 3-18

```
3 require_once "conexion.php";
4
5 $resultado = $conexion->query(
6     "SELECT
7         ambulancia.matricula,
8         ambulancia.marca,
9         ambulancia.modelo,
10        ambulancia.capacidad,
11        ambulancia.activa,
12        estado_ambulancia.nombre AS estado
13    FROM ambulancia
14    INNER JOIN estado_ambulancia
15        ON estado_ambulancia.id_estado_ambulancia = ambulancia.id_estado_ambulancia
16    ORDER BY ambulancia.matricula"
17 );
18
```

La ambulancia guarda id_estado_ambulancia, no el texto 'Disponible'. El JOIN recupera el nombre desde la tabla catálogo. Esto evita escribir estados diferentes por error y permite centralizarlos.

Alta

new-ambulance.php - validation and INSERT | lines 7-31

```
7 if ($_SERVER["REQUEST_METHOD"] === "POST") {
8     $matricula = strtoupper(trim($_POST["matricula"] ?? ""));
9     $marca = trim($_POST["marca"] ?? "");
10    $modelo = trim($_POST["modelo"] ?? "");
11    $capacidad = filter_input(INPUT_POST, "capacidad", FILTER_VALIDATE_INT);
12    $idEstado = filter_input(INPUT_POST, "id_estado_ambulancia", FILTER_VALIDATE_INT);
13
14    if (
15        $matricula === "" || $marca === "" || $modelo === "" ||
16        $capacidad === false || $capacidad < 1 ||
17        $idEstado === false || $idEstado === null || $idEstado < 1
18    ) {
19        $mensajeError = "Complete correctamente todos los campos.";
20    } else {
21        try {
22            $conexion->query(
23                "INSERT INTO ambulancia
24                (matricula, id_estado_ambulancia, marca, modelo, capacidad)
25                VALUES
26                ('$matricula', $idEstado, '$marca', '$modelo', $capacidad)"
27            );
28
29            header("Location: ambulance.php");
30            exit;
31        } catch (mysqli_sql_exception $error) {
```

Dato	Tratamiento
Matrícula	trim elimina espacios y strtoupper la normaliza a mayúsculas.
Capacidad	FILTER_VALIDATE_INT y mínimo 1.
Estado	Se valida como entero y la clave foránea confirma que exista.
Duplicado	El código 1062 permite mostrar un mensaje específico.

Por qué la matrícula no se edita

En esta base la matrícula es PRIMARY KEY y otras tablas pueden referenciarla. El formulario de edición la muestra como readonly y modifica marca, modelo, capacidad y estado. Cambiar una clave referenciada requiere una decisión explícita de diseño y reglas de actualización en cascada.

Eliminación

Antes de borrar una ambulancia se eliminan sus filas en ambulancia_equipamiento. Si un traslado la referencia, la clave foránea puede impedir el DELETE y el código muestra que la unidad está siendo utilizada.

8. Documentos: base de datos + archivos

Este módulo tiene dos tipos de persistencia: el archivo binario se mueve a documentos_subidos/ y MySQL guarda título, ruta y fecha. Las dos partes deben mantenerse sincronizadas.



cargar-documento.php - controles del archivo | líneas 13-31

```

13 if ($_SERVER["REQUEST_METHOD"] === "POST") {
14     $titulo = trim($_POST["titulo"] ?? "");
15     $archivo = $_FILES["documento"] ?? null;
16
17     $extensionesPermitidas = ["pdf", "doc", "docx", "txt", "jpg", "jpeg", "png"];
18     $tamanoMaximo = 10 * 1024 * 1024;
19
20     if ($titulo === "" || strlen($titulo) > 200) {
21         $mensajeError = "Ingresa un título de hasta 200 caracteres.";
22     } elseif ($archivo === null || $archivo["error"] !== UPLOAD_ERR_OK) {
23         $mensajeError = "Seleccione un archivo válido.";
24     } elseif ($archivo["size"] <= 0 || $archivo["size"] > $tamanoMaximo) {
25         $mensajeError = "El archivo debe pesar como máximo 10 MB.";
26     } else {
27         $extension = strtolower(pathinfo($archivo["name"], PATHINFO_EXTENSION));
28
29         if (!in_array($extension, $extensionesPermitidas, true)) {
30             $mensajeError = "El formato del archivo no está permitido.";
31         } else {

```

cargar-documento.php - movimiento y consulta preparada | líneas 43-55

```

43     $rutaParaBase = "documentos_subidos/" . $nombreGuardado;
44
45     if (!move_uploaded_file($archivo["tmp_name"], $rutaFisica)) {
46         $mensajeError = "No se pudo guardar el archivo seleccionado.";
47     } else {
48         try {
49             $guardarDocumento = $conexion->prepare(
50                 "INSERT INTO documento (titulo, ruta_archivo) VALUES (?, ?)"
51             );
52             $guardarDocumento->bind_param("ss", $titulo, $rutaParaBase);
53             $guardarDocumento->execute();
54             $guardarDocumento->close();
55         } catch (Exception $e) {

```

Control	Qué cubre	Qué falta para producción
Extensiones permitidas	Limita pdf, doc, docx, txt, jpg, jpeg y png.	Verificar MIME/contenido real, no solo extensión.
Tamaño máximo	Rechaza archivos mayores a 10 MB.	Coordinar también límites de PHP y servidor web.
Nombre generado	Evita colisiones y caracteres peligrosos.	Guardar fuera de la carpeta pública o controlar descargas.
Consulta preparada	Separa valores de la instrucción SQL.	Aplicar el mismo patrón al resto del proyecto.

Actualizar o eliminar un documento

La edición permite cambiar solo el título o reemplazar el archivo. Si la base se actualiza con éxito, el archivo anterior se elimina. Si el UPDATE falla, se borra el archivo nuevo para no dejar basura. La baja busca primero la ruta, elimina el registro y luego borra el archivo físico.

```

eliminar-documento.php - baja coordinada | líneas 16-49

16
17 try {
18     // Busca la ruta antes de eliminar el registro para poder borrar también el archivo.
19     $buscarDocumento = $conexion->prepare(
20         "SELECT ruta_archivo FROM documento WHERE id_documento = ?"
21     );
22     $buscarDocumento->bind_param("i", $idDocumento);
23     $buscarDocumento->execute();
24     $buscarDocumento->bind_result($rutaArchivo);
25     $documentoExiste = $buscarDocumento->fetch();
26     $buscarDocumento->close();
27
28     if (!$documentoExiste) {
29         http_response_code(404);
30         die("El documento no existe.");
31     }
32
33     $eliminarDocumento = $conexion->prepare(
34         "DELETE FROM documento WHERE id_documento = ?"
35     );
36     $eliminarDocumento->bind_param("i", $idDocumento);
37     $eliminarDocumento->execute();
38     $eliminarDocumento->close();
39
40     // basename impide que una ruta guardada pueda borrar archivos fuera de esta carpeta.
41     $rutaFisica = __DIR__ . DIRECTORY_SEPARATOR . "documentos_subidos"
42         . DIRECTORY_SEPARATOR . basename($rutaArchivo);
43
44     if (is_file($rutaFisica)) {
45         unlink($rutaFisica);
46     }
47 } catch (mysqli_sql_exception $error) {
48     http_response_code(500);
49     die("No se pudo eliminar el documento.");

```

basename como defensa

Al construir la ruta física para borrar, basename conserva solo el nombre final. Así una ruta manipulada en la base no puede dirigir unlink fuera de documentos_subidos.

Qué significa bind_param('ss', ...)

La consulta contiene signos ? como lugares reservados. bind_param vincula los valores: s significa string e i significa integer. MySQL recibe estructura y datos por separado, lo que reduce el riesgo de inyección SQL.

Firma	Tipos
bind_param('ss', \$titulo, \$ruta)	dos cadenas
bind_param('ssi', \$titulo, \$ruta, \$id)	dos cadenas y un entero

9. Equipamientos, funcionarios y traslados

Equipamientos: un buen uso de consultas preparadas

nuevo-equipamientos.php - alta parcial | líneas 7-28

```
7  if ($_SERVER["REQUEST_METHOD"] === "POST") {
8      $nombre = strtoupper(trim($_POST["nombre"] ?? ""));
9      $descripcion = trim($_POST["descripcion"] ?? "");
10     $activo = trim($_POST["activo"] ?? "");
11
12     if (
13         $nombre === "" || $descripcion === "" || !in_array($activo, ["0", "1"], true)
14     ) {
15         $mensajeError = "Complete correctamente todos los campos.";
16     } else {
17         try {
18             $activoNumero = (int) $activo;
19             $guardarEquipamiento = $conexion->prepare(
20                 "INSERT INTO equipamiento (nombre, descripcion, activo) VALUES (?, ?, ?)"
21             );
22             $guardarEquipamiento->bind_param("ssi", $nombre, $descripcion, $activoNumero);
23             $guardarEquipamiento->execute();
24             $guardarEquipamiento->close();
25
26             header("Location: equipamientos.php");
27             exit;
28         } catch (mysqli_sql_exception $error) {
```

El módulo valida nombre, descripción y un estado limitado a 0 o 1. El INSERT usa una consulta preparada con tipos ssi. Sin embargo, la interfaz solo permite alta y listado; todavía no administra la relación con ambulancias ni ofrece edición o baja.

Patrón reutilizable	El alta de equipamientos ya muestra el patrón recomendado para futuros módulos: validar una lista cerrada, convertir el tipo y ejecutar un INSERT preparado. Puede servir de base para corregir funcionarios y completar otros ABM.
Para cerrar el módulo	Implementación esperada
Ficha	SELECT por id_equipamiento y salida escapada.
Edición	GET identifica, POST valida y UPDATE preparado.
Baja	Definir baja lógica o revisar referencias antes de DELETE.
Asignación	Formulario sobre ambulancia_equipamiento con cantidad.

Funcionarios: inconsistencias concretas

nuevo-funcionario.php - variables y destino | líneas 55-66

```
55         // MySQL completa fecha_registro y activo con sus valores predeterminados.
56         $conexion->query(
57             "INSERT INTO funcionario
58             (id_persona, id_funcionario, id_tipo_funcionario)
59             VALUES
60             ($idPersona, '$idfuncionario', '$idtipofuncionario')"
61         );
62
63         // Vuelve al listado para mostrar el nuevo funcionario.
64         header("Location: funcionarios.php");
65         exit;
66     } catch (mysqli_sql_exception $error) {
```

nuevo-funcionario.php - action incorrecto | líneas 112-119

```
112         <p class="estado estado-critico" role="alert"><?= $mensajeError ?></p>
113         <?php endif; ?>
114
115         <!-- required ayuda al usuario; la validación definitiva igualmente se realiza en PHP. -->
116         <form class="formulario formulario-dos-columnas" action="nuevo-paciente.php" method="post">
117             <label for="nombres">
118                 Nombre/s
119                 <input id="nombres" type="text" name="nombres" required>
```

Hallazgo	Consecuencia
El form apunta a nuevo-paciente.php.	Al enviar, ejecuta el alta de paciente, no la de funcionario.
\$idfuncionario y \$idtipofuncionario no se asignan.	El INSERT no recibe valores válidos para el rol.
Redirige a funcionarios.php.	Ese archivo no existe; el listado real es funcionario.php.
Editar enlaza a modificarfuncionario.php.	El archivo no existe.
El botón eliminar no está en un formulario.	No envía POST ni un id al archivo de baja.

Corrección mínima	El formulario debe apuntar a nuevo-funcionario.php, solicitar o resolver id_tipo_funcionario, omitir id_funcionario para que AUTO_INCREMENT lo genere y redirigir a funcionario.php. Editar y eliminar necesitan destinos existentes y formularios POST con el id correcto.
-------------------	---

Traslados: modelo ambicioso, interfaz pendiente

pulso_base_de_datos.sql - entidad traslado | líneas 114-139

```
114 CREATE TABLE IF NOT EXISTS traslado (  
115     id_traslado INT AUTO_INCREMENT,  
116     id_paciente INT,  
117     matricula VARCHAR(20) NOT NULL,  
118     id_creador INT NOT NULL,  
119     id_conductor INT NOT NULL,  
120     id_acompanante INT,  
121     id_tipo_carga INT NOT NULL,  
122     id_tipo_prioridad INT NOT NULL,  
123     id_estado_traslado INT NOT NULL,  
124     descripcion_carga VARCHAR(255),  
125     origen VARCHAR(200) NOT NULL,  
126     destino VARCHAR(200) NOT NULL,  
127     salida_prevista DATETIME NOT NULL,  
128     salida_real DATETIME,  
129     llegada_real DATETIME,  
130     PRIMARY KEY (id_traslado),  
131     FOREIGN KEY (id_paciente) REFERENCES paciente(id_paciente),  
132     FOREIGN KEY (matricula) REFERENCES ambulancia(matricula),  
133     FOREIGN KEY (id_creador) REFERENCES funcionario(id_funcionario),  
134     FOREIGN KEY (id_conductor) REFERENCES funcionario(id_funcionario),  
135     FOREIGN KEY (id_acompanante) REFERENCES funcionario(id_funcionario),  
136     FOREIGN KEY (id_tipo_carga) REFERENCES tipo_carga(id_tipo_carga),  
137     FOREIGN KEY (id_tipo_prioridad) REFERENCES tipo_prioridad(id_tipo_prioridad),  
138     FOREIGN KEY (id_estado_traslado) REFERENCES estado_traslado(id_estado_traslado)  
139 ) ENGINE=InnoDB;
```

La tabla traslado relaciona paciente opcional, ambulancia, creador, conductor, acompañante, tipo de carga, prioridad y estado. También guarda origen, destino y tiempos previstos/reales. Es una entidad central que integra casi todo el sistema.

Rol	Por qué aparecen varias FK a funcionario
id_creador	Quién registró o planificó el traslado.
id_conductor	Quién conduce la ambulancia.
id_acompanante	Quién acompaña; puede ser NULL.
Estado actual	traslado.html declara que el módulo está pendiente y no ejecuta consultas. Para defenderlo, describan el diseño de base como trabajo realizado y el ABM como trabajo futuro; no lo presenten como funcional.

Otras inconsistencias de navegación

- El README enlaza a PASOS_PARA_PROBAR_CON_XAMPP_Y_MYSQL.md, pero ese archivo no está incluido.
- El panel muestra tarjetas solo para pacientes, documentos y ambulancias, aunque la barra superior enlaza también funcionarios y equipamientos.
- La pantalla de ingreso aclara que el inicio de sesión no forma parte de esta etapa.

10. Seguridad y calidad técnica

Una buena defensa no consiste en afirmar que todo es perfecto. Conviene demostrar que entienden qué protege el código, qué riesgo queda y cómo lo mejorarían.

Tema	Estado observado	Mejora recomendada
Inyección SQL	Pacientes y ambulancias interpolan texto dentro de SQL.	Usar prepare + bind_param en todos los valores.
Salida HTML	Listados y fichas aplican htmlspecialchars.	Mantener escape en cada salida variable.
Autenticación	No hay login, sesión ni roles.	Agregar autenticación y autorización por operación.
CSRF	Las operaciones POST no usan token.	Incluir token de sesión y validarlo antes de modificar.
Transacciones	Altas y bajas de varias tablas se ejecutan por separado.	BEGIN/COMMIT/ROLLBACK para todo o nada.
Archivos	Valida extensión y tamaño.	Validar MIME, permisos y almacenamiento no público.
Errores	Se ocultan detalles de conexión y se muestran mensajes simples.	Registrar errores técnicos en un log privado.
Integridad	Existen claves foráneas y UNIQUE.	Definir reglas ON DELETE/UPDATE según negocio.

Consulta directa vs. consulta preparada

Consulta directa actual	Consulta preparada recomendada
... VALUES ('\$nombres', '\$ci') Los datos forman parte del texto SQL.	... VALUES (?, ?) Los datos se vinculan por separado.
Una comilla en un apellido puede romper la consulta.	Una comilla permanece como parte del valor.
Puede abrir la puerta a inyección SQL.	Reduce ese riesgo y aclara los tipos.
Diferencia que suelen preguntar htmlspecialchars protege el HTML al mostrar; prepare/bind_param protege la consulta SQL al guardar o buscar. Son defensas para contextos distintos y se necesitan ambas.	

Plan de mejora priorizado

Prioridad	Cambio	Por qué primero
P0	Corregir action, variables y enlaces de funcionarios.	Hoy bloquean el caso de uso.
P0	Corregir la baja de paciente usando id_persona real y transacción.	Puede romper integridad de datos.
P1	Parametrizar consultas de pacientes y ambulancias.	Reduce inyección SQL y errores con comillas.
P1	Agregar autenticación, autorización y tokens CSRF.	Necesario si deja de ser un prototipo local.
P1	Validar MIME de documentos y revisar dónde se sirven.	Un upload requiere defensa por capas.
P2	Completar ABM de equipamientos y funcionarios.	Cierra módulos ya visibles en navegación.
P2	Implementar traslados usando el modelo existente.	Integra el resto de las entidades.
P3	Unificar nombres, helpers y plantillas repetidas.	Mejora mantenimiento y consistencia.

Qué significa que pase el chequeo de sintaxis

El comando de lint confirma que no hay errores de sintaxis PHP en los 20 archivos. No detecta archivos faltantes, variables indefinidas que solo aparecen al ejecutar un camino, errores de negocio, consultas incompatibles con los datos, ni vulnerabilidades. Por eso la revisión funcional sigue siendo necesaria.

Respuesta madura

El código puede ser sintácticamente válido y funcional en algunos módulos, pero todavía contener defectos lógicos. En esta entrega, funcionarios y la baja de pacientes son ejemplos claros.

11. Ejemplos para explicar en voz alta

Ejemplo A: registrar un paciente



- Si la CI ya existe, la restricción UNIQUE produce el código 1062 y se muestra un mensaje específico.
- Si la fecha está vacía, PHP envía NULL.
- MySQL completa fecha_registro y activo por DEFAULT.

Ejemplo B: buscar 'critico'

JavaScript normaliza tanto la consulta como el texto de cada fila: elimina diacríticos y convierte a minúsculas. Así critico coincide con Crítico. El filtro cambia la propiedad hidden y recalcula el contador; no consulta MySQL.

Ejemplo C: cargar protocolo.pdf



Si mover el archivo funciona pero el INSERT falla, el catch elimina el archivo recién creado. Esa compensación evita un archivo sin registro en la base.

Ejemplo D: abrir una ambulancia

El listado construye ficha-ambulancia.php?matricula=... usando urlencode. La ficha toma la matrícula desde GET, ejecuta un JOIN con el catálogo de estados y muestra 404 si no hay coincidencia.

12. Guion breve para la defensa

Apertura de 90 segundos

PULSO es un sistema web académico desarrollado sin framework con HTML, CSS, JavaScript, PHP y MySQL. La interfaz organiza módulos para pacientes, ambulancias, documentos, funcionarios, equipamientos y traslados. El navegador envía formularios por GET o POST; PHP valida, consulta o modifica MySQL y vuelve a generar HTML. La base separa persona de sus roles y usa claves foráneas para relacionar los datos. Los flujos más completos son pacientes, ambulancias y documentos. Como decisiones técnicas destacamos el uso de JOIN para reunir datos normalizados, insert_id para enlazar un alta de persona con paciente, escape HTML al mostrar y consultas preparadas en documentos y equipamientos. También identificamos limitaciones: faltan parametrizar consultas, usar transacciones, incorporar autenticación y completar funcionarios y traslados.

Demostración sugerida

Orden	Qué mostrar	Concepto que prueba
1	Panel y navegación.	Alcance y módulos.
2	Listado de pacientes.	SELECT, JOIN, escape y foreach.
3	Alta de paciente.	POST, validación, dos INSERT e insert_id.
4	Ficha y edición.	GET validado y dos UPDATE.
5	Carga de documento.	Archivos + consulta preparada.
6	Esquema SQL.	Claves, catálogos y relaciones.
7	Limitaciones conocidas.	Criterio técnico y próximos pasos.

Si algo falla durante la demo

- Distinguir si falla Apache/PHP, MySQL, la importación del esquema o una ruta de archivo.
- Confirmar que conexion.php usa la base pulso_entrega.
- No improvisar: indicar el archivo implicado, el flujo esperado y la limitación conocida.

13. Preguntas probables y respuestas

1. ¿Por qué separar persona y paciente?

Para no duplicar datos personales y permitir que una persona tenga distintos roles. Paciente conserva solo información clínica y referencia a persona.

2. ¿Qué hace un JOIN?

Combina filas relacionadas de dos tablas. En pacientes reúne datos personales con estado, patología y fecha de registro.

3. ¿Por qué usar POST para eliminar?

Porque elimina o modifica estado. GET debería ser una lectura y puede activarse accidentalmente por enlaces, caché o previsualizaciones.

4. ¿Qué hace insert_id?

Devuelve la clave autogenerada por el último INSERT en esa conexión. Se usa para relacionar el nuevo paciente con su persona.

5. ¿Qué diferencia hay entre id y matrícula?

Un id suele ser numérico y autogenerado; la matrícula es una clave natural elegida como PRIMARY KEY de ambulancia.

6. ¿Qué hace htmlspecialchars?

Escapa caracteres para que un valor se muestre como texto en HTML y no se interprete como etiquetas o script.

7. ¿Qué protege bind_param?

Separa datos y sintaxis SQL, reduciendo inyección y problemas con caracteres especiales.

8. ¿Por qué validar dos veces?

HTML ayuda al usuario; PHP es la frontera de confianza; MySQL garantiza integridad final.

9. ¿Qué significa una clave foránea?

Una columna solo puede apuntar a una fila existente en la tabla referenciada.

10. ¿Por qué hay una tabla ambulancia_equipamiento?

Porque la relación es muchos a muchos y además necesita guardar la cantidad.

11. ¿Qué problema tiene eliminar_paciente.php?

Supone que id_paciente e id_persona son iguales, algo que el modelo no garantiza.

12. ¿Qué módulo usa mejor SQL seguro?

Documentos y equipamientos usan prepare y bind_param para sus operaciones principales.

13. ¿El sistema tiene login?

No. La pantalla inicial aclara que la autenticación quedó fuera de esta etapa.

14. ¿Traslados funciona?

La base está modelada, pero la vista declara el módulo pendiente y no hay ABM conectado.

15. ¿Qué agregarían primero?

Corregir funcionarios y la baja de pacientes; luego parametrizar SQL, transacciones, autenticación y CSRF.

14. Banco de autoevaluación

Intenten responder sin mirar y luego comparen con la clave. Las preguntas mezclan reconocimiento, comprensión y aplicación.

- 1. Ordená las capas: MySQL, navegador y PHP.
- 2. ¿Qué atributo HTML hace que un input llegue con una clave a \$_POST?
- 3. ¿Por qué required no alcanza como única validación?
- 4. ¿Qué diferencia hay entre PRIMARY KEY y FOREIGN KEY?
- 5. ¿Qué devuelve fetch_all(MYSQLI_ASSOC)?
- 6. ¿Por qué el alta de paciente hace dos INSERT?
- 7. ¿Qué valor crea MySQL automáticamente en paciente.fecha_registro?
- 8. ¿Qué evita htmlspecialchars y en qué momento actúa?
- 9. ¿Qué evita una consulta preparada y en qué momento actúa?
- 10. ¿El filtro de pacientes consulta la base cada vez que se escribe?
- 11. ¿Por qué la matrícula está readonly al editar?
- 12. ¿Dónde se guarda un documento y qué guarda MySQL?
- 13. Nombrá dos controles actuales del upload y dos controles faltantes.
- 14. Explicá el defecto de la baja de pacientes.
- 15. ¿Qué tablas participan en la relación N a N de equipamientos?
- 16. ¿Qué módulos están realmente completos o casi completos?
- 17. ¿Qué diferencia hay entre pasar el lint y funcionar correctamente?
- 18. ¿Qué dos operaciones deberían estar dentro de una transacción en el alta de paciente?

Clave breve

- 1. Navegador -> PHP -> MySQL, y la respuesta vuelve en sentido inverso.
- 2. name.
- 3. Puede saltarse; PHP debe validar datos no confiables.
- 4. La primaria identifica; la foránea referencia otra fila.
- 5. Un arreglo asociativo de filas.
- 6. Persona y paciente están normalizados en tablas distintas.
- 7. CURRENT_TIMESTAMP.
- 8. XSS/salida HTML al renderizar.
- 9. Inyección SQL al ejecutar consultas.
- 10. No, filtra las filas ya descargadas.
- 11. Es la clave primaria y puede estar referenciada.
- 12. Binario en documentos_subidos; título, ruta y fecha en MySQL.
- 13. Actuales: extensión y tamaño. Faltan: MIME real y control de acceso/ubicación.
- 14. Usa id_paciente como si fuera id_persona.
- 15. ambulancia, equipamiento y ambulancia_equipamiento.
- 16. Pacientes, ambulancias y documentos, con observaciones.
- 17. Lint solo verifica sintaxis, no lógica ni dependencias.
- 18. INSERT persona e INSERT paciente.

15. Prompts sugeridos para NotebookLM

Después de cargar este PDF, estos pedidos ayudan a convertirlo en distintos materiales de estudio.

Resumen progresivo	Explicá PULSO en tres niveles: 30 segundos, 3 minutos y 10 minutos. Mantené separadas las funcionalidades reales de las pendientes.
Quiz oral	Haceme 15 preguntas de defensa, una por vez. Esperá mi respuesta, corregime con evidencia del documento y aumentá gradualmente la dificultad.
Tarjetas	Creá 30 flashcards pregunta-respuesta sobre PHP, MySQL, GET/POST, JOIN, claves, validación, escape, prepared statements y módulos de PULSO.
Video	Escribí un guion de video de 6 minutos con escenas, narración y sugerencias visuales. Empezá por la arquitectura y terminá con limitaciones y mejoras.
Presentación	Generá una presentación de 10 diapositivas para la defensa: problema, solución, arquitectura, base, pacientes, ambulancias, documentos, seguridad, estado real y cierre.
Simulacro crítico	Actuá como tribunal técnico. Preguntá por decisiones cuestionables del proyecto, especialmente SQL directo, transacciones, baja de pacientes, funcionarios y uploads.
Trazado de datos	Elegí un caso de uso y describí el recorrido exacto de los datos desde el formulario hasta MySQL y de vuelta al HTML.

16. Glosario y machete final

Término	Definición
ABM / CRUD	Alta, baja, modificación y consulta; Create, Read, Update, Delete.
Cliente	El navegador que muestra la interfaz y envía solicitudes.
Servidor	Apache/PHP, que procesa la solicitud y construye la respuesta.
Request	Solicitud HTTP con URL, método y, a veces, datos.
Response	Respuesta HTTP con estado y contenido.
GET	Método habitual para leer o identificar un recurso.
POST	Método usado aquí para enviar cambios.
SQL	Lenguaje para consultar y modificar la base.
JOIN	Operación que combina tablas relacionadas.
PRIMARY KEY	Identificador único de una fila.
FOREIGN KEY	Referencia a una clave de otra tabla.
UNIQUE	Restricción que impide duplicados.
NULL	Ausencia de valor conocido; no equivale a texto vacío.
DEFAULT	Valor aplicado por MySQL si el INSERT lo omite.
Array asociativo	Colección accedida con nombres de clave.
Excepción	Error que puede capturarse con try/catch.
Prepared statement	SQL precompilado con valores vinculados por separado.
XSS	Ejecución de contenido no confiable como HTML/JavaScript.
SQL injection	Manipulación de una consulta mediante datos no separados.
Transacción	Grupo de operaciones que confirma todo o revierte todo.

Machete de una página

Si preguntan...	Responder con...
Arquitectura	Navegador -> PHP -> MySQL; PHP devuelve HTML.
Alta paciente	POST, validar, INSERT persona, insert_id, INSERT paciente, redirect.
Listado paciente	SELECT + INNER JOIN + fetch_all + foreach + escapar.
Edición	GET identifica; POST valida; dos UPDATE.
Base	PK identifica, FK relaciona, UNIQUE evita duplicados, DEFAULT completa.
JavaScript	Filtro local, normaliza tildes y mayúsculas, no toca MySQL.
Documentos	Archivo físico + ruta en base; extensión, tamaño, nombre único y prepare.
Seguridad	Escape para HTML; prepare para SQL; faltan auth, CSRF y transacciones.
Estado real	Pacientes, ambulancias y documentos son el núcleo; resto parcial o pendiente.
Defecto clave	La baja de paciente confunde id_paciente con id_persona.

Cierre recomendado

El proyecto demuestra el circuito completo de una aplicación web tradicional y un modelo relacional razonable. La siguiente etapa es consolidar seguridad, transacciones y consistencia, y completar los módulos parciales sobre los patrones que ya funcionan.

Archivos fuente revisados

Código PHP, HTML y CSS del directorio Pulso-entrega; esquema principal pulso_base_de_datos.sql; README.md y GUIA_PARA_ESTUDIAR.md como documentación auxiliar. Los fragmentos oscuros de este PDF se extrajeron directamente de los archivos indicados y conservan sus números de línea.

Fin de la guía · Preparada para estudio individual, simulacros de defensa y uso como fuente en NotebookLM.